Introduction To Algorithms

Week 2 – Growth and Big-O Intuition:

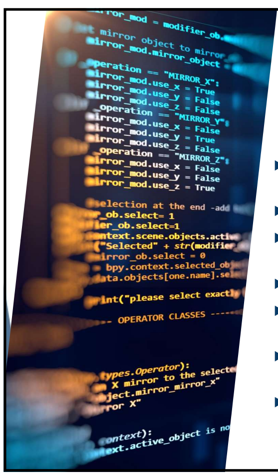
"When Working Code Starts to Slow Down"

1

Terms to Remember

Term	Brief Meaning
Performance analysis	Studying how an algorithm behaves as resources & input size change
Space complexity	How memory use changes as input grows
Time complexity	How the amount of work or time changes as input grows
Input size	The amount of data the algorithm processes
Big-O	A notation for describing growth behavior
Best case	The easiest or fastest likely situation for an algorithm
Worst case	The hardest or slowest likely situation for an algorithm
Average case	Expected behavior across typical inputs
Exact algorithm	Designed to produce an exact correct result
Approximate algorithm	Designed to produce a good-enough result under constraints
Randomized algorithm	Uses randomness as part of its process
Explainability	Ability to describe how or why an algorithm produced a result

2



What We Will Use Today

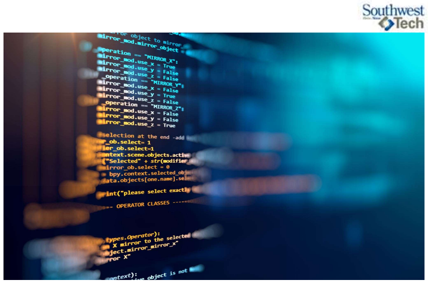
- correct, understandable, efficient
- space and time analysis
- input size and growth behavior
- Big-O vocabulary
- best, worst, and average case
- timing evidence and its limits
- algorithm selection and explanation

3

What We Will "Touch On" and Revisit Later

- Formal mathematical notation
- Recursion-based complexity
- Nested recursion
- Memory optimization
- GPU/CPU details
- Rrandomized and approximate algorithms in deeper AI/data contexts

4



Review Coding Lab

5

Review: What Lab 01 Taught Us

Last week, we focused on:

- precise rules
- expected results
- actual results
- edge cases
- revision based on evidence

6

Today's Question

What happens when the amount of data gets larger?

7

From "Works" To "Scales"

Correctness asks:

- Does the algorithm produce the expected result?

Growth asks:

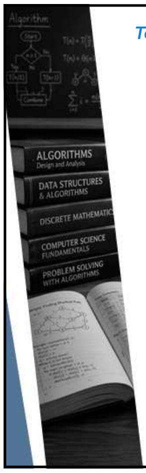
- How does the work change as input grows?

8

What Counts as Success Today

- compare two approaches
- collect timing evidence
- describe growth cautiously
- use basic Big-O vocabulary
- identify a limitation of your evidence

9



Textbook Review

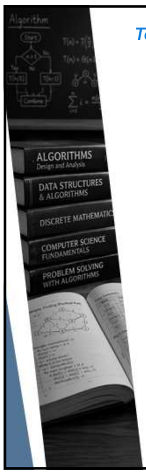
Southwest Tech

Algorithm Quality

The reading describes strong algorithms as:

- Correct
- Understandable
- Efficient

10



Textbook Review

Southwest Tech

Performance Analysis

Performance analysis asks:

- How much time might this take?
- How much memory might this use?
- What happens as input size grows?

11

Textbook Review

Southwest Tech

Space And Time

Two common analysis types:

- Space analysis: memory growth
- Time analysis: work or runtime growth

12

Textbook Review

Space Complexity

Space complexity asks how memory use changes.

Patterns can be:

- Flat
- Converging
- Diverging

13

Textbook Review

Time Complexity

Time complexity asks:

- Can this scale?
- What work does the algorithm repeat?
- What happens with larger datasets?
- What resources are involved?

14

Textbook Review

Input Size

Input size is the amount of data the algorithm works on.

Examples:

- number of list items
- number of records
- number of characters
- number of graph nodes

15

Performance Analysis Foundations

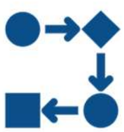
16

Growth Pattern

Growth pattern means:

- how work changes
- as input size changes

Small inputs can hide the pattern.



17

Timing Is Evidence

Timing can show:

- what happened in a run
- on this machine
- with this code
- under these conditions

18

Southwest Tech

Timing Can Be Noisy

Timing can vary because of:

- background activity
- hardware differences
- setup code
- number of trials
- input construction

19

Big-O Is Pattern Language

Big-O describes growth behavior.

It does not mean:

- exact seconds
- one timing result
- every machine behaves identically

20

Southwest Tech

What Counts As Work?

Work may include:

- comparisons
- loop iterations
- lookups
- calculations
- recursive calls
- memory writes

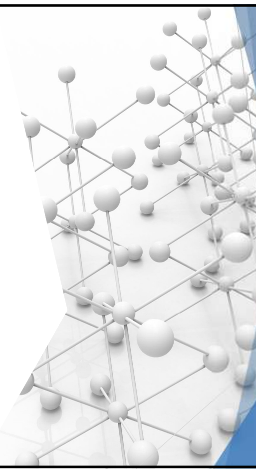
21

Space And Time Tradeoff

Sometimes an approach uses more memory to save time.

Example:

- list scanning: less structure, more repeated checking
- set lookup: extra structure, faster membership checks



22

CPU, GPU, And Memory

Algorithms use machine resources:

- CPU: general processing
- GPU: many parallel operations
- memory: stored data and working space

23

Week 2 Practical Rule

- What is the input size?
- What work repeats?
- Does memory grow?
- Does timing suggest a pattern?

24

Southwest Tech

Big-O Vocabulary

Giving a label to growth behavior

25

Southwest Tech

Big-O Notation

Big-O gives a label to growth behavior.

Examples:

- O(1) “O of one” or “constant time”
- O(log n) “O of log n” or “O of logarithmic time”
- O(n) “O of n” or “linear time”
- O(n2) “O of n squared”
- O(n log n) “linearithmic (lin-ear-ith-mic)”

26

Southwest Tech

Constant Time: O(1)

O(1) means the work stays about the same as input grows.

Example idea:

- get one known item
- check one stored value
- return a cached count

27

Linear Time: O(n)

O(n) means work grows with the input size.

Example idea:

- look at each item once
- count matching records
- find a maximum with one pass

28

Quadratic Time: O(n2)

O(n2) often appears when work is nested.

Example idea:

- compare each item to every other item
- check many possible pairs
- nested loops over the same input

29

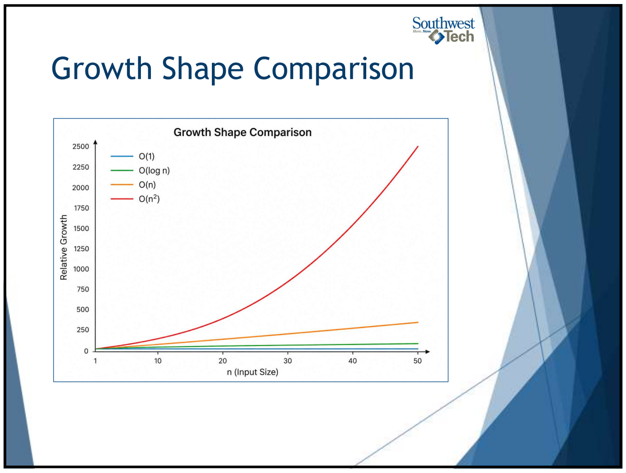
Logarithmic Time: O(log n)

O(log n) often appears when the search space is repeatedly divided.

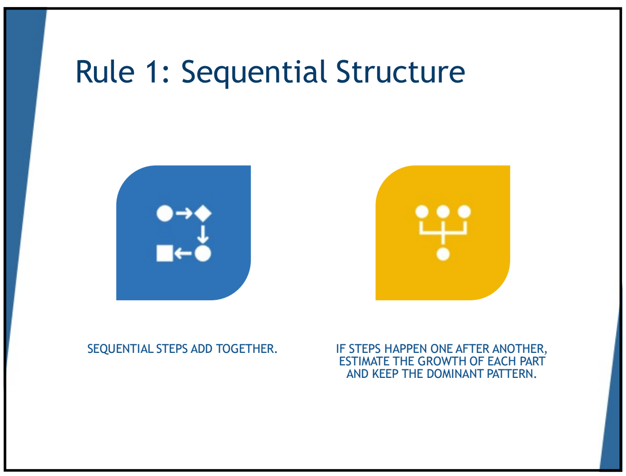
Example idea:

- binary search
- cut the remaining possibilities in half
- stop when the target is found

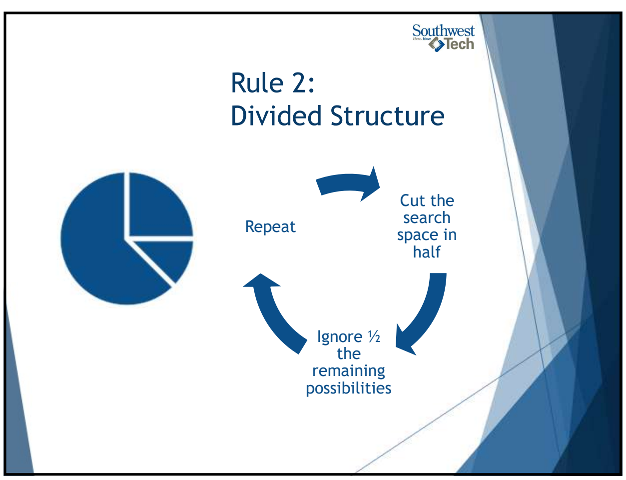
30


Southwest Tech
Growth Shape Comparison
Growth Shape Comparison
O(1)
O(log n)
O(n)
O(n²)
Relative Growth
2500
2250
2000
1750
1500
1250
1000
750
500
250
0
1
10
20
30
40
50
n (Input Size)

31


Rule 1: Sequential Structure
SEQUENTIAL STEPS ADD TOGETHER.
IF STEPS HAPPEN ONE AFTER ANOTHER, ESTIMATE THE GROWTH OF EACH PART AND KEEP THE DOMINANT PATTERN.

32


Southwest Tech
Rule 2:
Divided Structure
Cut the search space in half
Ignore ½ the remaining possibilities
Repeat

33

Rules 3 & 4: Recursion and Nested Recursion

Recursion:
- a function calls itself

Nested recursion:
- recursive work happens inside more recursive work

34

Rule 5: Ignore Constant Multiples

Big-O focuses on growth shape.

For basic analysis, constant multiples are usually ignored.

Example:
- 3n still grows like n
- 100n still grows like n

35

Timing Evidence vs Big-O Reasoning

Timing asks:
- What happened in this experiment?

Big-O asks:
- What growth pattern should we expect?

Use both carefully.

36

Break

10 Minute “Human”
bio-break

37

Southwest Tech

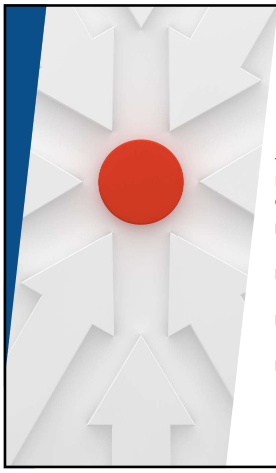
Estimating and Selecting

38

Best, Worst, And Average Case

1. Best case: easiest input
2. Worst case: hardest input
3. Average case: typical input

39



Why Case Matters

The same algorithm may look different depending on:

- Where the target appears
- Whether the target exists
- Whether input is already organized
- Whether data has duplicates

40

Selecting An Algorithm

- Correctness
- Understandability
- Efficiency
- Memory use
- Data size
- Explainability

41

Validating Algorithms

Validation depends on the algorithm type:

- Exact
- Approximate
- Randomized

42

What Evidence Does NOT Prove

A timing table does not prove:

- Every machine will match
- Every input will behave the same
- The algorithm is always best
- The code has no bugs

43

Explainability

44

Southwest Tech

Explainability

Explainability means you can describe:

- What the algorithm does
- Why the approach fits
- What evidence supports it
- What limits remain

45

Good Growth Explanation

A responsible explanation says:

- What changed
- What evidence showed
- What Big-O label likely fits
- What the evidence does not prove

46

Example

"In my timing table, Approach A increased more as input size grew. This matches the idea of repeated scanning, so linear or worse growth may be involved. However, my timing results are limited by machine noise and only four input sizes."

47

Demo Scenario

Problem:

Compare two ways to check whether values appear in a collection:
- *Manual list lookup with a loop*
- *Set membership lookup with `in`*

The demo uses the same lookup task for both approaches, then repeats the task across larger input sizes.

Evidence:

1. A timing table across several input sizes.
2. A formatted comparison summary that interprets the pattern.

Goal

Check whether values appear in a collection

48

Predict The Work

List lookup may:

- Check many values
- Repeat scanning
- Grow with collection size

Set membership may:

- Use a structure built for lookup
- Avoid visible full scanning

49

Demo Evidence

Watch the table for:

- Input size
- List lookup time
- Set lookup time
- Change across rows
- Reminder about noisy timing

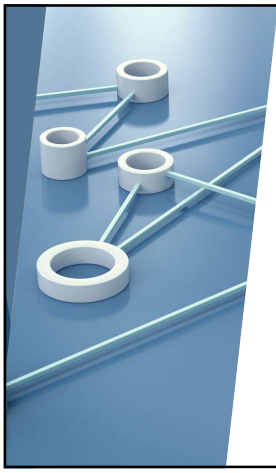
50

Demo Explanation

Demo explanation pattern:

- Both approaches were correct
- Timing changed as input grew
- List lookup grew more noticeably
- Set lookup changed less in this demo
- Exact numbers may vary

51



AI-Assisted Complexity Review

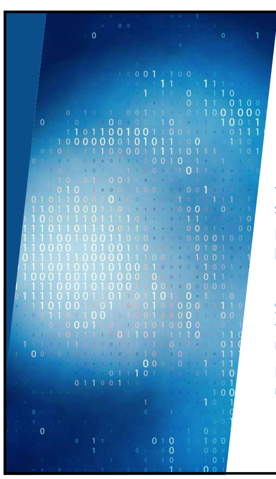
52

AI Can Help Review Complexity

AI can help you:

- Analyze likely time complexity
- Analyze likely space complexity
- Identify repeated work
- Explain Big-O in plain language
- Suggest alternative approaches

53



Useful AI Prompt Pattern

"Analyze this Python function for likely time and space complexity.

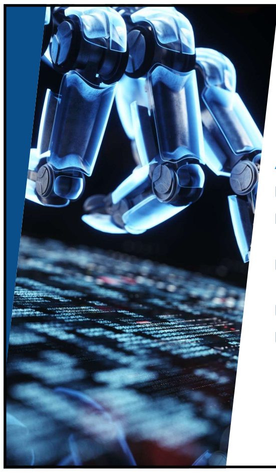
Explain the reasoning in beginner-friendly language.

Identify repeated work.

Suggest one alternative if the current approach may not scale.

Do not rewrite the code unless I ask."

54



Verify The AI Review

After AI responds, check:

- Does it match the code?
- Does it identify the repeated work?
- Does it match the timing pattern?
- Did it overclaim?
- Can you restate it yourself?

55

Demo Add-On: Console Color

After the plain demo works, add light console color.

The color helps highlight:

- section headings
- passed tests
- failed tests
- important boundary cases

56

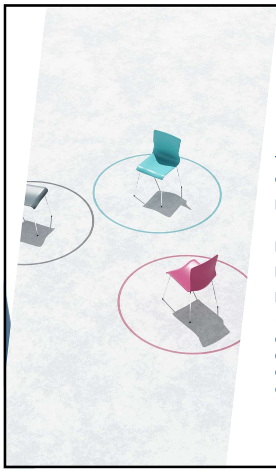
From Demo to Lab

In Lab 02, you will compare two approaches.

Your work must include:

- Two implementations
- At least four input sizes
- Timing table
- Chart or comparison table
- Growth explanation
- Limitation note

57



Walkthrough Support

The walkthrough artifact can help you:

- Identify both approaches
- Predict repeated work
- Decide what to time
- Explain the difference

If you use it, you must explain the pattern in your own words, what you changes, and your decisions:

58

README Evidence

Your README should show:

- approaches compared
- input sizes
- timing evidence
- chart or comparison table
- likely growth pattern
- timing limitation
- AI-use note, if applicable

59

Wrap up

60

What To Carry Forward

Performance analysis asks:

- Does it work?
- Is it understandable?
- How does it grow?
- What evidence supports the claim?

61

Lab 02 Success Check

- Compares two correct approaches
- Uses increasing input sizes
- Reports measured timing
- Explains likely growth
- Names a limitation

62

Next Step

1

For Lab 02:

- choose a comparison
- implement or attempt both approaches
- collect timing evidence
- build a table or chart
- explain the pattern cautiously

63

How To Use The Textbook For Next Week's Reading

For next week, read for structure choice.

Focus on:

- What each structure is for
- What operations each structure makes easier
- What operations each structure makes harder
- How complexity tables can guide decisions
- Why tabular, matrix, and abstract structures appear in algorithm work

Do not memorize every operation cost. Use the tables like reference maps.

64

Thank you!

Have Fun!

65